

1. Answer the questions in the spaces provided. For multiple choice questions indicate all correct choices (as A, B, etc) in the given boxes ☐ clearly.
2. Each MCQ could have zero, one or multiple correct choices and each correctly marked choice is awarded two marks (+2) while you loose half-mark (-1/2) for each incorrect choice marked. Python questions (Q4) are five marks each.

Maximum marks: 40. Total Time: 45m.

1. Mark all those code snippets from the choices below that print **banana** as the answer.

☐ ☐ ☐ ☐ ☐ ☐

- A. `let fruits = ["apples", "banana"]; console.log( fruits[1] );`
- B. `const fruit = "apples"; fruit = "banana"; console.log( fruit );`
- C. `let fruit = "apples"; fruit = "banana"; console.log( fruit );`
- D. `var fruit = "apples"; fruit = "banana"; console.log( fruit );`
- E. `var fruit = "apples"; var fruit = "banana"; console.log( fruit );`

2. Answer the following questions.

- (a) The below code snippet upon execution on the call stack

☐ ☐ ☐ ☐ ☐ ☐

```
const elClicks=document.getElementById("clicks");
elClicks.textContent = "100";
```

- A. instantly re-renders the content on browser to reflect the new value 100
 - B. only updates the text in id=clicks element of HTML DOM tree but does not render it immediately**
 - C. adds a function to the task queue for updating the DOM tree
 - D. sends a request to the browser to update the corresponding value using web API.
- (b) The functions of (event loop, call stack) are to
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
 A. (store variables, hold pending tasks) B. (modify HTML DOM, monitor browser events)
C. (schedule from task queue, execution scheduled code) D. (asynchronously run timer-counting like jobs, hold callbacks waiting for processing)
 - (c) What happens when the call stack is empty?
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
 A. event loop terminates **B. event loop pushes tasks from task queue to call stack**
 C. program terminates D. event loop restarts listening for user actions
 - (d) Which of these lead to an unresponsive UI?
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
 A. event loop listening for actions on the UI **B. while(var i<1000000){i++;}** **C. any compute-heavy function on the call stack** D. event loop moving callbacks to task queue
 - (e) What is the cause of such unresponsiveness?
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
 A. the event-loop failing to run and all events are missed **B. single thread of call stack being occupied while tasks are waiting in queue** C. the call-stack is emptied and has none to execute **D. having slow running functions running on the call stack** E. the task queue is full and unable to take anymore callbacks.

3. Answer questions below on the following snippet of code from the lecture.

```
1. function bumpStatus(text){
2.     statusTextEl.textContent = text;
   /*
3.     sleep(1000) // thread blocking pause
4.     statusText.textContent = defaultStatus;
   */
5.     clearTimeout(bumpStatus._t);
```

```

6.      bumpStatus._t = setTimeout( () => {
          statusTextEl.textContent = "Listening!"; }, 500 );
      }

```

(a) What does the `clearTimeout` function call on line 5 achieve?

☐ ☐ ☐ ☐ ☐ ☐ ☐

A. Ensures callbacks from line 6 are not needlessly accumulating in the task queue.

B. Intimates the web API to discard the timeout from previous call. C. Ensures the timeouts do not go beyond the 500ms limit from line 6. D. Removes the previous callback pushed into the task queue from earlier run of line 6.

(b) If we were to replace lines 5 and 6 with lines 3 and 4, which of the following are true?

☐ ☐ ☐ ☐ ☐ ☐ ☐

A. During the execution of sleep function, the UI becomes unresponsive. B. Any events like key-presses occurring during sleep execution are missed. **C. Callbacks from any button presses during sleep execution will be pushed to task queue waiting execution.**

4. Answer the following question for Python.

(a) Write a python program to print a diamond pattern on the screen using '*' character. Take the total number of rows as input (assume odd) and make sure the pattern is symmetric.

(b) What does the following function (**fn**) do? Describe in less than 5 sentences.

```
def fn(s: str):  
    if not s:  
        return s  
    result = [s[0]]  
    for ch in s[1:]:  
        if ch != result[-1]:  
            result.append(ch)  
    return ''.join(result)
```

END OF BOOKLET

EXTRA SPACE